

JavaScript Validation

What is Form Validation in JavaScript?

- ✓ **Form validation** is the process of ensuring that user input in a form is **correct, complete, and in the right format** before it is sent to the server.
- ✓ In **JavaScript**, this is done on the **client-side** (in the browser) to give quick feedback and reduce invalid submissions.
- ✓ The browser itself prevents submission if the field is empty.
But often we want **custom messages or complex rules**, and that's where **JavaScript** comes in.

Ways to access form elements

Using document.forms

```
obj = document.forms["form_name"]["element_name"].value;
```

- Accesses the form using its **name**
- Then accesses the input using its **name**
- Clear and safe when working with multiple forms

Using form and element names directly

```
obj = document.form_name.element_name.value;
```

- Shorter syntax
- Works only if both **form name** and **element name** exist
- Considered **old-style** and less recommended today

Using getElementById (Best method)

```
obj = document.getElementById("id").value;
```

- Accesses the element using its **id**
- **Most modern and recommended approach**
- Independent of form names

```
obj = document.forms["form_name"]["element_name"].value;
```

or

```
obj = document.form_name.element_name.value;
```

or

```
obj = document.getElementById(id).value;
```

Example : Event (onsubmit)

```
<head>
<script>
function fun()
{
    // METHOD1
    var obj = document.f1.t1.value;

    // METHOD2
    //var obj = document.forms["f1"]["t1"].value;

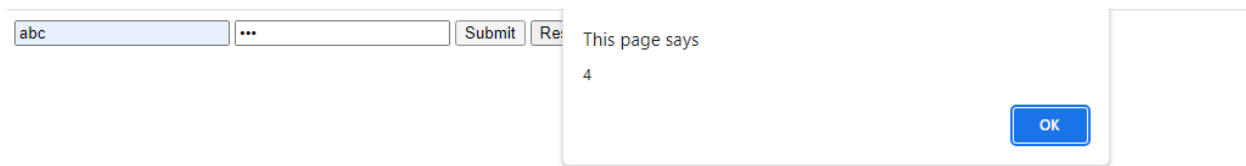
    // METHOD3
    //var obj = document.getElementById("i1").value;
    alert(obj);
}
</script>
</head>
<body>
<form name="f1" onsubmit="fun()">
    <input type="text" name="t1" id="i1" />
    <input type="submit"/>
</form>
</body>
```

Example: To find Form Length

```
<html>
<head>
<script type="text/javascript">
function fun()
{
    var obj = document.forms["f1"];
    alert(obj.length);
}
</script>
</head>
<body>
<form name="f1" onsubmit="fun()">
    <input type="text" name="t1"/>
</form>
```

```
<input type="password"/>
<input type="submit"/>
<input type="reset"/>
</form>
</body>
</html>
```

Output: (on clicking submit button.)

A screenshot of a web browser showing a form with two input fields. The first field contains 'abc' and the second is empty. There are 'Submit' and 'Reset' buttons. An alert box is displayed over the form, containing the text 'This page says' and the number '4'. The alert box has an 'OK' button.

Example: Alert will pop up with text “Enter Something” if input field is empty on clicking submit button.

```
<html>
<body>
<script type="text/javascript">
function fun(demo)
{
    var val=demo.t1.value;
    if(val.length==0)
    {
        alert("Enter something");
        return false;
    }
    alert(val);
}
</script>
<form action="#" onsubmit="fun(this)">
    <input type="text" maxlength="10" placeholder="enter Name" id="t1"/>
    <input type="submit" value="submit"/>
</form></body></html>
```

Output:

The image shows two states of a web form. In the top state, the text input field contains 'abc', the submit button is disabled, and an alert dialog box is open showing 'This page says abc'. In the bottom state, the text input field has the placeholder 'enter Name', the submit button is active, and an alert dialog box is open showing 'This page says Enter something'.

Example (another method of above example)

```
<html>
<head>
</head>
<body>
<script type="text/javascript">
function fun()
{
    val=document.getElementById("t1").value;
    if(val=="")
    {
        alert("Enter something");
        return false;
    }
    alert(val);
}
</script>
<form action="#">
    <input type="text" maxlength="10" placeholder="enter Name" id="t1"/>
    <input type="submit" value="submit" onclick="fun()"/>
</form>
```

Regular Expression(Regex)

A regular expression is a pattern of characters.

Syntax of Regex:

/pattern/modifier(s)

test() : This method is called using pattern object and returns true if string is a part of pattern.

syntax:

RegExpObject.test(string)

The String is Required. It is to be searched. It returns true if it finds a match, otherwise false.

Pattern	Description	Possible Output / Example Match
i	Case-insensitive matching	/abc/i → ABC, Abc
^	Matches start of string	^Hello → Hello world
\$	Matches end of string	world\$ → Hello world
\.	Matches literal dot	a\.b → a.b
\\	Matches backslash	\\ → \
*	Zero or more occurrences	zo* → z, zoo
+	One or more occurrences	zo+ → zoo
?	Zero or one occurrence	colou?r → color, colour
x y	Matches x or y	z wood → z, wood
{n}	Exactly n times	o{2} → oo
{n,}	At least n times	o{2,} → oo, oooo
{n,m}	Between n and m times	o{1,3} → o, oo, ooo
[abc]	Any one of a, b, c	plain → a
[^abc]	Any char except a, b, c	plain → p, l, n
[a-z]	Lowercase letters	hello → h, e
[A-Z]	Uppercase letters	HELLO → H, E
[A-z]	A to z (includes symbols)	A, Z, [, _, a
[+-]?	Optional + or -	+5, -5, 5
/^[^xy]\$/	One character, not x or y	a ✓, x ✗
/^[^xy]+\$/	One or more chars, not x or y	abc ✓, axc ✗

Pattern	Description	Possible Output / Example Match
/^[^xy]{3}\$/	Exactly 3 chars, not x or y	abc ✓ , ab ✗

Pattern	Equivalent Character Set	Description	Possible Output / Example Match
\d	[0-9]	Any digit	123 → 1, 2, 3
\D	[^0-9]	Any non-digit	a9# → a, #
\w	[A-Za-z0-9_]	Word character (letters, digits, underscore)	user_1 → u, _, 1
\W	[^A-Za-z0-9_]	Non-word character	@#! → @, #, !

🚦 Brackets [abc] specifies matches for the characters inside the brackets.

- [abc] Any of the characters a, b, or c
- [A-Z] Any character from uppercase A to uppercase Z
- [a-z] Any character from lowercase a to lowercase z
- [A-z] Any character from uppercase A to lowercase z

🚦 Brackets [^abc] specifies matches for any character NOT between the brackets.

- [^abc] Not any of the characters a, b, or c
- [^A-Z] Not any character from uppercase A to uppercase Z
- [^a-z] Not any character from lowercase a to lowercase z
- [^A-z] Not any character from uppercase A to lowercase z

/^[0-9]\$/ - This regex ensures that the input is **exactly one digit**, with no extra characters before or after.

/^[A-Z]\$/ - This regex ensures that the input contains exactly **one uppercase letter**, and nothing else.

Examples

Example 1: String must contain 'abc' anywhere in whole string

```
<head>
<script>
function fun()
{
  pat = /abc/;
```

```
u = document.getElementById("t1").value;
if(pat.test(u))
{
    alert("PERFECT");
}
else{
    alert("String must contain 'abc' anywhere in whole string")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
```

```
//abc - true
//abewr = false
//agrtabc = true
//ab = false
//Abc = false
```

Note:

pat = **/abc/i**;

"i" makes string case insensitive. ABC,aBc,ABcxyx11 are allowed.

Example 2: Must contain 'a' or 'b' or 'c' or any combination of these three letters even 'abc' is also allowed.

```
<html>
<head>
<script>
function fun()
{
    pat = /[abc]/;
    u = document.getElementById("t1").value;
    if(pat.test(u))
    {
        alert("PERFECT");
    }
    else{
```

```
    alert("Must contain 'a' or 'b' or 'c' or any combination of these three characters even 'abc' is  
also allowed")  
}  
}  
</script>  
</head>  
<body>  
<input type="text" id="t1"/>  
<input type="submit" onclick="fun()"/>  
</body>  
</html>  
  
//ffazsd - true  
//a = true  
//grtabc = true  
//ab = true  
//abc = true  
//df = false  
// xvkjhytr = false
```

Example 3: Not any of the characters a, b, or c. String may contain 'a' or 'b' or 'c' or any combination of these three letters, but with other alphabets/digits/special characters only.

```
<html>  
<head>  
<script>  
function fun()  
{  
pat = /^[^abc]/;  
u = document.getElementById("t1").value;  
if(pat.test(u))  
{  
    alert("PERFECT");  
}  
else{  
    alert("Not any of the characters a, b, or c. String may contain 'a' or 'b' or 'c' or any  
combination of these three letters, but with other alphabets/digits/special characters only.")  
}  
}  
</script>  
</head>
```



```
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed - ae,the,met,ant,cat,this.

Not Allowed: a,abc,ca,cbbca

Example 4: Digit must be present in string.

```
<html>
<head>
<script>
function fun()
{
pat = /[0-9]/;
u = document.getElementById("t1").value;
if(pat.test(u))
{
alert("PERFECT");
}
else{
alert("Digit must be present in string.")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples

Allowed- 11,123,ae1,11the,112@,a12sd@

Not Allowed: a,abc@, a@sdn

Example 5: Only Digits are not allowed in string. String must contain alphabet or special characters with digits.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat = /^[^0-9]/;
u = document.getElementById("t1").value;
if(pat.test(u))
{
alert("PERFECT");
}
else{
    alert("Only Digits are not allowed in string. String must contain alphabet or special
characters with digits.")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed- ae1,11the,112@,a12sd@

Not Allowed: 1,12,111

Example 6: String must contain x or y.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat = /(x|y)/;
u = document.getElementById("t1").value;
if(pat.test(u))
{
alert("PERFECT");
}
}
```

```
else{
    alert("String must contain x or y.")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed - x,y,xys,x1,x@asd

Not Allowed: 11, @df,sdf1

Example 7: String must start with A.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
    pat = /^A/;
    u = document.getElementById("t1").value;
    if(pat.test(u))
    {
        alert("PERFECT");
    }
    else{
        alert("String must start with A.")
    }
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed - A1, Ant, Awesome

Not Allowed: all,a123,a@ddd,sdA

Example 8: String must contain B. * means 0 or more. So occurrence of a can be 0 or more.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat = /Ba*/;
u = document.getElementById("t1").value;
if(pat.test(u))
{
alert("PERFECT");
}
else{
    alert("String must contain B. * means 0 or more. So occurrence of a can be 0 or more.")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed - B1, Back, taBle,B@eee

Not Allowed: ba, table, @ta11 (B must be capital)

Example 9: String must end with "t".

```
<html>
<head>
<script type="text/javascript">
```

```
function fun()
{
pat = /t$/;
u = document.getElementById("t1").value;
if(pat.test(u))
{
    alert("PERFECT");
}
else{
    alert("String must end with t.")
}
}
</script>
</head>
<body>
<input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Examples :

Allowed - Bat, Best, 11t,@t

Not Allowed: top, table1, @caT

Example 10: Must contain only 10 digits.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat = /^d{10}$/; // \d means digits. {10} means only 10 characters. ^ means starts with, $
means ends with.
u = document.getElementById("t1").value;
if(pat.test(u))
{
    alert("PERFECT");
}
else{
    alert("Must contain only 10 digits.")
}
```

```
}  
}  
</script>  
</head>  
<body>  
<input type="text" id="t1"/>  
<input type="submit" onclick="fun()"/>  
</body>  
</html>
```

Examples :

Allowed – 1234567891

Not Allowed: 1234567erd,1234

Example 11: Must contain only letters and at least one letter or more are allowed.

```
<html>  
<head>  
<script type="text/javascript">  
function fun()  
{  
pat = /^[A-Za-z]+$; // [A-Za-z] means letters. + means one or more characters. ^ means  
starts with, $ means ends with. Only letters are allowed.  
u = document.getElementById("t1").value;  
if(pat.test(u))  
{  
alert("PERFECT");  
}  
else{  
alert("Must contain only letters and at least one letter or more are allowed")  
}  
}  
</script>  
</head>  
<body>  
<input type="text" id="t1"/>  
<input type="submit" onclick="fun()"/>  
</body>  
</html>
```

Examples :

Allowed – abc,xyz,a
Not Allowed: abc11,1x

Example: Write HTML form accepting an integer having four digits. Input should not accept characters of letters and special symbols.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat=/^\d{4}$/;
u = document.getElementById("t1").value;

if((pat.test(u)))
{
alert("Perfect");
}
else{
    alert("Only 4 digit pin is allowed. (No letters/ no special characters)");
}
}
</script>
</head>
<body>
Pin <input type="text" id="t1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Example:

Write a JS to validate username and password.

Password: Length must be of 6 to 12 characters.

Username: Should not start with _, @ and any number.

Both must not be blank

```
<!DOCTYPE html>
<html>
<head>
<title>Username & Password Validation</title>
```

```
<script>
function fun()
{
    // Username should NOT start with _, @, or number
    var userPattern = /^[_@0-9]/;

    // Password length: 6 to 12 characters
    var passPattern = /^[w]{6,12}$/;;

    var uname = document.getElementById("t1").value;
    var pass = document.getElementById("p1").value;
    var msg = document.getElementById("test");

    msg.style.color = "red";

    // Check blank
    if (uname.length === 0 || pass.length === 0)
    {
        msg.innerHTML = "Username and Password must not be blank";
    }
    // Validate username
    else if (userPattern.test(uname))
    {
        msg.innerHTML = "Username should not start with _, @, or a number";
    }
    // Validate password
    else if (!passPattern.test(pass))
    {
        msg.innerHTML = "Password length must be between 6 and 12 characters";
    }
    else
    {
        msg.style.color = "green";
        msg.innerHTML = "Welcome " + uname;
    }
}
</script>
</head>
<body>
NAME :
<input type="text" id="t1"><br><br>
PASS :
```



```
<input type="password" id="p1"><br><br>
<input type="button" value="Submit" onclick="fun()">
<h3 id="test"></h3>
</body>
</html>
```

Example: Design a login form using JS. Following validation in password field, Minimum length of password must be of 8 letters and it must have some special characters.

```
<html>
<head>
<script type="text/javascript">
function fun()
{
pat=/\W+/;
  u = document.getElementById("p1").value;
  if(u.length <= 8){
    alert("Password must be greater than 8");
  }
  else if(!pat.test(u))
  {
    alert("Password must be strong...Add at least one special char");
  }
  else
  {
    alert("Password Entered: " + u);
  }
}
</script>
</head>
<body>
<input type="password" id="p1"/>
<input type="submit" onclick="fun()"/>
</body>
</html>
```

Email Regex

```
/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/
```

Example: lju.test@yahoo.co.in

Regex Part	Meaning	Example
^	Start of email	—
\w+	Name (letters/numbers/_)	rahul, amit123
([.-]?\w+)*	Optional . or - in name	rahul.kumar, amit-singh
@	Must have @	@
\w+	Domain name	gmail, yahoo, gov
([.-]?\w+)*	Subdomain (optional)	mail.nic
(\.\w{2,3})+	Domain extension	.com, .in, .co.in
\$	End of email	

Valid Emails

rahul.kumar@gmail.com
amit-singh@yahoo.co.in
sneha123@gov.in
info@mail.nic.in

Invalid Emails

.rahul@gmail.com ✗ starts with dot
rahul@gmail ✗ no domain extension
rahul@gmail.india ✗ extension too long
rahul@@gmail.com ✗ double @

Example:

```
<body>  
<form>  
  <label>Email:</label><br><br>
```

```

<input type="text" id="email" placeholder="rahul.kumar@gmail.com">
<br><br>
<button type="button" onclick="validateEmail()">Submit</button>
</form>

<p id="result"></p>

<script>
function validateEmail() {
  const email = document.getElementById("email").value;
  const regex = /^\\w+([.-]?\\w+)*@\\w+([.-]?\\w+)*\\.\\w{2,3}+$/;

  document.getElementById("result").innerText =
    regex.test(email) ? "Valid Email" : "Invalid Email";
}
</script>
</body>

```

Another Simple email

Example: Email Address /^\\w+@\\w+ (\\.\\w{2,3})+\$/

Example: lju@gmail.com }

Regex	Description	Matches (Examples)	Does Not Match
/abc/	Must contain substring abc anywhere	abc, xabc, 123abcxyz	ab, axyz
/abc/i	Case-insensitive match for abc	ABC, aBc, ABcxyz	axyz
/[abc]/	Must contain at least one of a, b, or c	a, cab, xyzab	xyz, ddf
/^[abc]/	Must contain at least one character other than a, b, or c	cat, the, 123d	a, abc, cbbca
/[0-9]/	Must contain at least one digit	a1, 123, the2	abc, @xyz
/[^0-9]/	Must contain at least one non-digit	a1, @12, xyz9	1, 123, 999
/(x y)/	Must contain x or y	x, y, x1y2, asdx, y123	11, zzz
/^A/	Must start with A	Apple, A1, Awesome	a123, bA
/Bo*/	Must contain B followed by zero or more o	B, Bo, Boo, Born	bo, table

Regex	Description	Matches (Examples)	Does Not Match
/t\$/	Must end with t	Bat, Best, 123t	top, table1
/^d{10}\$/	Exactly 10 digits	1234567890	1234, abc1234567
/^[A-Za-z]+\$	Only letters (uppercase/lowercase), at least one required	abc, XYZ, a	abc123, 1a, @b
/^d{4}\$/	Exactly 4 digits	1234, 9876	123, 12a4
/^[_@0-9]/	Username must start with _, @, or a digit	_abc, @user, 1name	user1, abc_1, hello123
/\W+/	Must contain at least one special character	abc@123, pass!word	password123, abcde
/^w+([.-]?w+)*@w+([.]?w+)*(\.w{2,3})+\$/	Email validation	test@gmail.com, abc.xy@domain.co.in	abc@com, @gmail.com
/^[0-9]\$/	Exactly one digit (0–9)	0, 5, 9	12, a, 99
/^[A-Z]\$/	Exactly one uppercase letter	A, Z, M	AB, a, 1

Local Storage and Session Storage

What is Web Storage?

JavaScript provides **Web Storage API** for saving key–value data directly in the browser.

Two main types:

- **localStorage** → Data stored permanently (until manually cleared).
- **sessionStorage** → Data stored only for the current browser tab/session.

Both:

- Store data as **string key–value pairs**.
- Have a storage limit (~5–10 MB, much larger than cookies).
- Are synchronous (simple to use).

Local Storage

- Persists even after the browser is closed/reopened.
- Good for saving **preferences, settings, theme, cached data**.

Example:

```
// Save
```

```
localStorage.setItem("username", "ABC");
```

```
// Retrieve
```

```
let user = localStorage.getItem("username");
```

```
console.log(user); // "ABC"
```

```
// Update
```

```
localStorage.setItem("username", "ABC");
```

```
// Delete one item
```

```
localStorage.removeItem("username");
```

```
// Clear all
```

```
localStorage.clear();
```

Session Storage

- Data lasts **only while the browser tab is open**.
- Once the tab is closed, data is gone.
- Good for **temporary data** like form input, step-by-step processes.

Example:

// Save

```
sessionStorage.setItem("cart", "3 items");
```

// Retrieve

```
let cart = sessionStorage.getItem("cart");  
console.log(cart); // "3 items"
```

// Remove

```
sessionStorage.removeItem("cart");
```

// Clear all

```
sessionStorage.clear();
```

Method	Description	Syntax	Example
setItem(key, value)	Stores a key–value pair. Value is always saved as a string .	storage.setItem("key", "value")	localStorage.setItem("username", "Alice");
getItem(key)	Retrieves the value for the given key. Returns null if not found.	storage.getItem("key")	localStorage.getItem("username"); // "Alice"
removeItem(key)	Removes a specific item from storage (only that key).	storage.removeItem("key")	localStorage.removeItem("username");
clear()	Removes all items from storage.	storage.clear()	localStorage.clear();

Step 1: Open your HTML page

- Open your HTML file in the browser.

Step 2: Open Developer Tools

- **Right-click anywhere on the page**
- Click **Inspect**
OR
- Press **F12**
- Or **Ctrl + Shift + I** (Windows)
- Or **Cmd + Option + I** (Mac)

Step 3: Go to the Application tab

- In Developer Tools, click **Application**
- If not visible, click the » **(double arrow)** and select **Application**

Step 4: Find Storage section (Left panel)

Under **Storage**, you will see:

- **Local Storage**
- **Session Storage**

Step 5: Click your website

- Click the URL shown under:
 - **Local Storage** → to see localStorage
 - **Session Storage** → to see sessionStorage

Login Demo (Save/Show/Clear)

```
<!DOCTYPE html>
<html>
<body>
  <h2>Login Demo</h2>
  <input type="text" id="username" placeholder="Enter username">
  <button onclick="saveUser()">Save</button>
  <button onclick="showUser()">Show</button>
  <button onclick="clearUser()">Clear</button>

  <p id="output"></p>

  <script>
    function saveUser() {
      let user = document.getElementById("username").value;
      localStorage.setItem("username", user); // persists
      sessionStorage.setItem("username", user); // temp
    }

    function showUser() {
      let localUser = localStorage.getItem("username");
      let sessionUser = sessionStorage.getItem("username");
      document.getElementById("output").innerHTML =
        "Local: " + localUser + "<br>Session: " + sessionUser;
    }

    function clearUser() {
```

```
localStorage.clear();
sessionStorage.clear();
document.getElementById("output").innerHTML = "Cleared!";
}
</script>
</body>
</html>
```

Login Demo

By Clicking on **save** button

The first screenshot shows the 'Application' tab with 'Local storage' selected. A table lists a single entry: 'username' with the value 'abc'.

Key	Value
username	abc

The second screenshot shows the 'Application' tab with 'Session storage' selected. A table lists two entries: 'IsThisFirstTime_Log_From_LiveServer' with the value 'true', and 'username' with the value 'abc'.

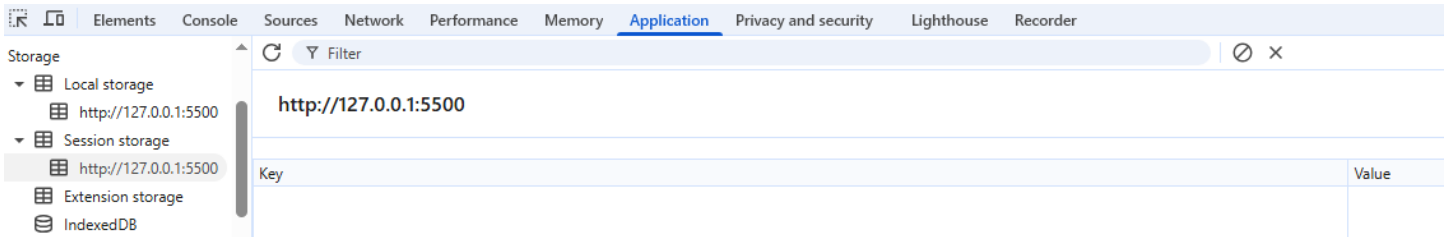
Key	Value
IsThisFirstTime_Log_From_LiveServer	true
username	abc

By Clicking on **show** button

Login Demo

Local: abc
Session: abc

By Clicking on **clear** button



PB examples

Create a small HTML page where:

You store two fields in sessionStorage:

username = "lj"

email = "lj@example.com"

Retrieve and display the stored username in the console.

Remove only the email field from sessionStorage, then print the remaining data in the console.

```
<!DOCTYPE html>
<html>
<head>
  <title>SessionStorage Example</title>
</head>
<body>

<script>
  // Store data in sessionStorage
  sessionStorage.setItem("username", "lj");
  sessionStorage.setItem("email", "lj@example.com");

  // Retrieve and display username
  console.log("Username:", sessionStorage.getItem("username"));

  // Remove only the email field
  sessionStorage.removeItem("email");

  // Print remaining data
  console.log("Remaining sessionStorage data:");
  console.log("Username:", sessionStorage.getItem("username"));
  console.log("Email:", sessionStorage.getItem("email")); // null
</script>

</body>
```

</html>

Create a small HTML page where:

You store three fields in localStorage:

product = "Laptop"

price = 50000

discount = 10

Retrieve and display the stored product in the console.

Remove the discount field from localStorage, then print the updated data in the console.

```
<!DOCTYPE html>
<html>
<head>
  <title>localStorage Example</title>
</head>
<body>

<script>
  // 1. Store data in localStorage
  localStorage.setItem("product", "Laptop");
  localStorage.setItem("price", 50000);
  localStorage.setItem("discount", 10);

  // 2. Retrieve and display product
  console.log("Product:", localStorage.getItem("product"));

  // 3. Remove discount field
  localStorage.removeItem("discount");

  // 4. Print updated data
  console.log("Updated localStorage data:");
  console.log("Product:", localStorage.getItem("product"));
  console.log("Price:", localStorage.getItem("price"));
  console.log("Discount:", localStorage.getItem("discount")); // null
</script>

</body>
</html>
```

Create a form with three fields:
product,price,discount and a Save button.

Requirements:

1. When the user clicks Save, store the entered values into localStorage.
2. After saving, display a message:Data saved in local storage!
3. Add another button Remove Discount that removes only the discount field from localStorage.
4. After removal, display the remaining data in the console.

```
<!DOCTYPE html>
<html>
<head>
  <title>LocalStorage Form</title>
</head>
<body>

<h3>Product Details</h3>

<form onsubmit="return saveData()">
  Product:
  <input type="text" id="product" required><br><br>

  Price:
  <input type="number" id="price" required><br><br>

  Discount:
  <input type="number" id="discount" required><br><br>

  <input type="submit" value="Save">
</form>

<br>

<button onclick="removeDiscount()">Remove Discount</button>

<script>
function saveData() {
  // Store values in localStorage
  localStorage.setItem("product", document.getElementById("product").value);
  localStorage.setItem("price", document.getElementById("price").value);
  localStorage.setItem("discount", document.getElementById("discount").value);

  // Display message
```

```
    alert("Data saved in local storage!");

    return false; // prevent page refresh
}

function removeDiscount() {
    // Remove only discount
    localStorage.removeItem("discount");

    // Display remaining data in console
    console.log("Remaining localStorage data:");
    console.log("Product:", localStorage.getItem("product"));
    console.log("Price:", localStorage.getItem("price"));
    console.log("Discount:", localStorage.getItem("discount")); // null
}
</script>

</body>
</html>
```

Asynchronous JavaScript

Introduction to Promises (.then(), .catch(), .finally())

What is Asynchronous JavaScript?

JavaScript normally executes code **synchronously**, meaning one line runs after another.

However, some operations such as **API calls, timers, file loading, and database access** take time to complete.

Asynchronous JavaScript allows these time-consuming operations to run in the background **without blocking the rest of the program**.

Why Asynchronous Programming is Needed

Without asynchronous handling:

- The browser would freeze while waiting
- User experience would be poor
- Long tasks would block UI interaction

Asynchronous programming ensures:

- Better performance
- Responsive applications
- Non-blocking execution

What is a Promise?

A **Promise** is an object that represents the **future result** of an asynchronous operation.

A Promise can be in one of three states:

1. **Pending** – operation is still running
2. **Fulfilled** – operation completed successfully
3. **Rejected** – operation failed

Promise Creation Syntax

```
let promise = new Promise((resolve, reject) => {  
  // asynchronous operation  
});
```

- `resolve(value)` → called when the task is successful
- `reject(error)` → called when the task fails

Promise Handling Methods

.then() – Success Handler

Runs when the Promise is **fulfilled**.

.catch() – Error Handler

Runs when the Promise is **rejected**.

.finally() – Always Executes

Runs **regardless of success or failure**.

Syntax:

```
let promise = new Promise((resolve, reject) => {  
  // async task  
  if (success) {  
    resolve(result);  
  } else {  
    reject(error);  
  }  
});  
promise  
  .then(result => console.log(result))  
  .catch(error => console.log(error))  
  .finally(() => console.log("Process finished"));
```

Complete Promise Example

```
let order = new Promise((resolve, reject) => {  
  let success = true;  
  
  setTimeout(() => {  
    if (success) {  
      resolve("Order completed");  
    } else {  
      reject("Order failed");  
    }  
  }, 2000);  
});  
  
order  
  .then(result => console.log(result))  
  .catch(error => console.log(error))  
  .finally(() => console.log("Process finished"));
```

What is setTimeout() and When to Use It?

Purpose of setTimeout()

setTimeout() is used to:

- Delay execution of code
- Simulate real-time delays (API calls, loading, waiting)

Syntax

```
setTimeout(function, delayInMilliseconds);
```

Example

```
setTimeout(() => {  
  console.log("Executed after 2 seconds");  
}, 2000);
```

When to Use setTimeout()

Use setTimeout() when:

- Simulating asynchronous behavior
- Delaying execution
- Showing loaders or messages after a delay
- Testing Promise behavior

Async / Await Syntax

What is async / await?

async / await is a **modern way to write asynchronous JavaScript**.

It is built **on top of Promises** and makes asynchronous code look like **normal synchronous code**.

Why async / await is Used

Promises using .then() and .catch() work well, but:

- They can become **hard to read**
- Chained .then() calls look messy

async / await solves this by:

- Making code **cleaner**
- Making logic **easier to understand**
- Improving **error handling**

async Keyword

Purpose

The async keyword is used **before a function** to indicate that:

- The function will work with asynchronous code
- The function **always returns a Promise**

Syntax

```
async function functionName() {  
  // async code  
}
```

await Keyword

Purpose

The await keyword is used to:

- Pause execution until a Promise is resolved

- Retrieve the resolved value of the Promise
await can be used **only inside an async function**.

Syntax

```
let result = await promise;
```

Promise in async/await

```
return new Promise((resolve,reject) => {  
  // async work  
});
```

It means:“This function will not give the result immediately. It promises to give the result later.”

```
new Promise(...)
```

Creates a Promise object

A Promise represents a future value

async/await works only with Promises, not with callbacks. To use async/await, convert callback-based code into a Promise.

What is try...catch in async/await?

When using async/await, **errors are handled using try...catch** instead of .catch().

- try → code that may succeed or fail
- catch → runs if an error occurs

Syntax

```
async function functionName() {  
  try {  
    let result = await someAsyncFunction();  
    // success code  
  } catch (error) {  
    // error handling  
  }  
}
```

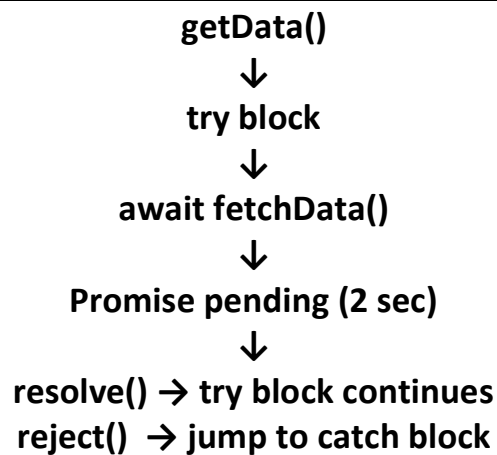
Example

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      let success = true; // change to false to test error
```



```
    if (success) {  
        resolve("Data received");  
    } else {  
        reject("Error while fetching data");  
    }  
}, 2000);  
});  
}
```

```
async function getData() {  
    try {  
        let result = await fetchData(); // wait for Promise  
        console.log(result);           // success  
    } catch (error) {  
        console.log("Error:", error); // error handling  
    }  
}  
getData();
```



PB Examples

Example

Output? `Promise.resolve().then(()=>console.log('A')); console.log('B');`

- `console.log('B')` runs **synchronously**, so it executes immediately.
- `Promise.resolve().then(...)` schedules its code, which runs **after** the current Execution.

Output:

B A

So above syntax is roughly equivalent to:

```
new Promise(resolve => resolve())
  .then(() => console.log('A'));
```

Example

Write a function that returns a Promise which resolves a number after 1 second. Use a loop with `async/await` to get values from 1 to 3. Print the doubled result of each value.

```
<!DOCTYPE html>
<html>
<body>

<script>
/* Function that returns a Promise
   Resolves the given number after 1 second */
function getNumber(num) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(num);
    }, 1000);
  });
}

/* Async function using loop and await */
async function processNumbers() {
  for (let i = 1; i <= 3; i++) {
    let value = await getNumber(i); // wait for promise
    console.log(value * 2);         // print doubled value
  }
}
```

```
}

processNumbers();
</script>
</body>
</html>
```

- `getNumber(num)`
 - returns a Promise
 - resolves the number after **1 second**
- `processNumbers()`
 - uses `async/await`
 - loop runs from **1 to 3**
 - waits 1 second for each number
 - prints the **doubled value**

Prompt the user to enter 3 numbers (you can use `prompt()` in browser). For each number, return a Promise that resolves after 1 second. Print the triple of each entered number.

```
<body>
<script>
/* Function that returns a Promise
   Resolves the number after 1 second */
function processNumber(num) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(num * 3); // triple the number
    }, 1000);
  });
}

/* Async function to handle user input */
async function getUserNumbers() {
  for (let i = 1; i <= 3; i++) {
    let input = prompt("Enter number " + i + ":");
    let number = Number(input);

    let result = await processNumber(number); // wait 1 second
    console.log("Triple:", result);
  }
}
```

```
getUserNumbers();  
</script>  
</body>
```

- User is prompted **3 times** to enter numbers
- Each number is passed to a Promise
- Promise resolves **after 1 second**
- The **triple of each number** is printed in the console

Fetch API

The **Fetch API** is a modern JavaScript interface used to **send HTTP requests** and **receive responses** from external APIs or servers.

It works asynchronously and returns a **Promise**.

HTTP GET Request using Fetch API

A **GET request** is used to **retrieve data** from a server.

Syntax

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

Example: GET Request with Fetch API

```
fetch("https://dog.ceo/api/breeds/image/random")
  .then(response => response.json())
  .then(data => {
    console.log("Image URL:", data.message);
    console.log("Status:", data.status);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

Explanation

- `fetch(url)` → sends HTTP GET request
- `response` → server response object
- `.json()` → converts JSON response to JavaScript object
- `.then()` → handles successful response
- `.catch()` → handles errors

Error Handling in Fetch API

Important Note

Fetch **does not automatically fail** on HTTP errors (404, 500).
You must check the response status manually.

Example with Proper Error Handling

```
fetch("https://dog.ceo/api/breeds/image/random")
  .then(response => {
    if (!response.ok) {
      throw new Error("Failed to fetch data");
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Error:", error.message);
  });
```

Concept	Purpose
fetch()	Send HTTP GET request
.json()	Parse JSON response
.then()	Handle success
.catch()	Handle errors
response.ok	Check HTTP status

PB Example

Create a simple web page that allows users to fetch and display a random dog image from an external API. The page should include:

1. A heading that indicates the purpose of the page (e.g., "Random Dog Image").
2. A button that, when clicked, sends a request to the Dog CEO API (<https://dog.ceo/api/breeds/image/random>) to fetch a random dog image.
3. An image element that displays the fetched dog image dynamically when the user clicks the button.
4. The solution should use JavaScript's fetch API and async/await syntax to handle the API request.

```
<body>
  <h2>Random Dog Image</h2>
  <button onclick="getDog()">Get Dog</button>
  <br><br>
  <img id="dogImg" width="300">
```

```
<script>
  function getDog() {
    fetch("https://dog.ceo/api/breeds/image/random")
      .then(response => response.json())
      .then(data => {
        document.getElementById("dogImg").setAttribute("src", data.message);
        console.log("Status:", data.status);
      })
      .catch(error => {
        console.log("Error:", error);
      });
  }
} </script>
</body>
```

- Displays a **Random Dog Image**
- When the user clicks the **Get Dog** button
- It fetches a random dog image from an **external API**
- Shows the image on the page